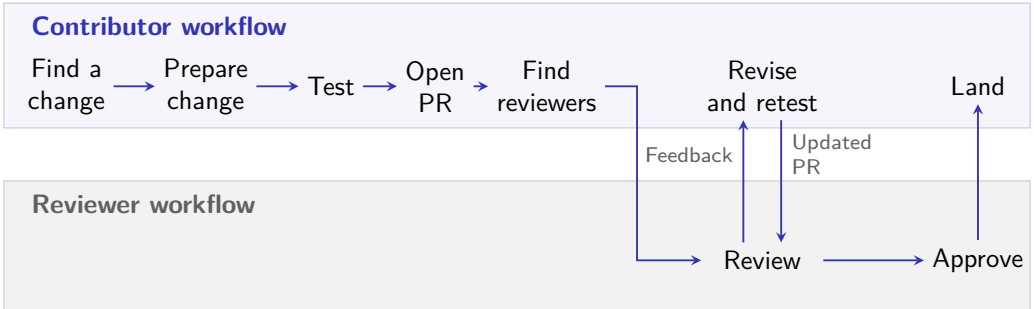
So You Want to Contribute to LLVM?

Justin Fargnoli

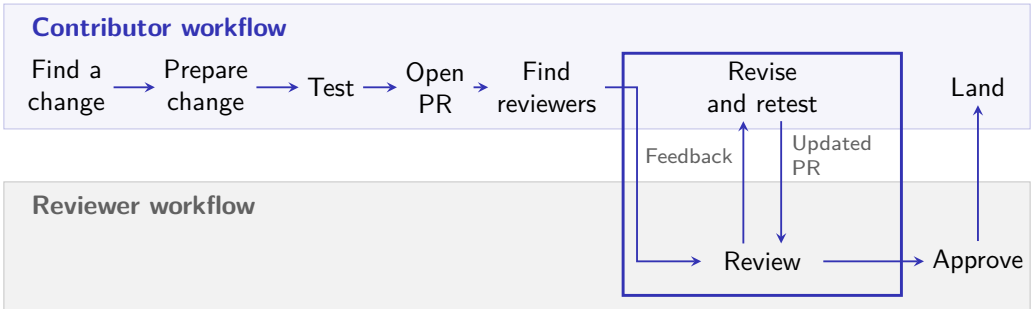
NVIDIA

2026 LLVM Developers' Meeting

The Contribution Process



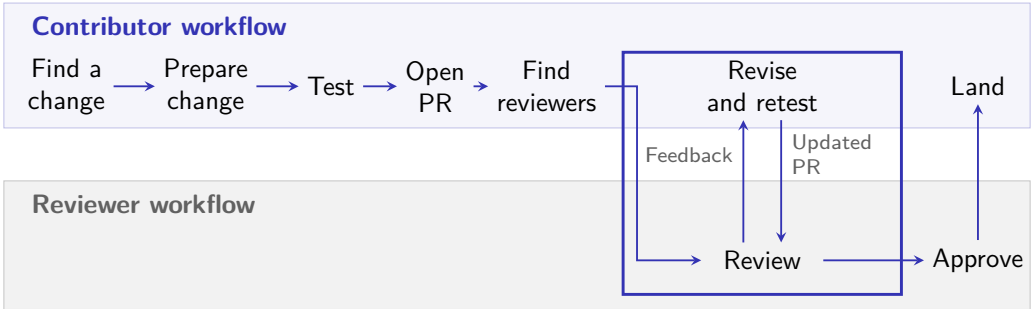
The Contribution Process



"One round of review can take two or three weeks."

"It can take months to get a PR reviewed and landed."

The Contribution Process



- Limited reviewer bandwidth can prolong review cycles.
- This talk focuses on working within that constraint.

How to Avoid Prolonged Reviews

Work with the LLVM community to get your code reviewed and landed.

- Put up a change that's ready for review.
- Find and get the attention of the right reviewers.
- Respond to reviewer feedback.

A Change That's Ready for Review

- Keep it small and self-contained.
- Write a clear PR description.
- Well-tested.
  - Positive and negative test cases.
  - Pre-commit testing workflow.
  - Alive2 proofs.
  - Compile-time tracker results.
  - LLVM Opt Benchmark results.
- CI is passing.
- Review the diff yourself.
  - Ask yourself: Why this approach? Why wasn't an alternative viable?

The PR Description

"Commit messages should communicate briefly what the change does, ... emphasize why a change is being made and provide useful context."

LLVM Developer Policy: commit messages (excerpt)

- Explain the problem and who benefits.
- Explain the design choices and alternatives.
- Link a reproducer or other evidence.

Find Relevant Reviewers

Find people who know the code and will challenge your change.

- Start with `Maintainers.md`.
- Use `git log` and `git blame`; find recent PRs and their reviewers.
- Request review or `@mention` them in a PR comment.

Getting the Attention of Reviewers

When a reviewer goes quiet, ping them.

"Ping the patch. If it is urgent, provide reasons why it is important to you to get this patch landed and ping it every couple of days. If it is not urgent, the common courtesy ping rate is one week."

LLVM Code-Review Policy: speed and reciprocity (excerpt)

> `Ping @reviewerA @reviewerB`
>
> `Ping 2x @reviewerA @reviewerB`

Getting the Attention of Reviewers

When a reviewer goes quiet, ping them.

"Ping the patch. If it is urgent, provide reasons why it is important to you to get this patch landed and ping it every couple of days. If it is not urgent, the common courtesy ping rate is one week."

LLVM Code-Review Policy: speed and reciprocity (excerpt)

> `Ping @reviewerA @reviewerB`
>
> `Ping 2x @reviewerA @reviewerB`

Get other people reviewing the code.

- A non-expert's review can draw an expert's attention.
- Keep seeking the expert's approval before merging.

Addressing Review Feedback

Reviewer	Contributor
"Could you use UpperCamelCase here?"	"The formatter changed that code. I didn't modify it."
"This belongs in type legalization."	"Good point. Do you mean vector legalization or integer legalization?"

Addressing Review Feedback

Reviewer	Contributor
"Could you use UpperCamelCase here?"	~~"The formatter changed that code. I didn't modify it."~~ "Done."
"This belongs in type legalization."	~~"Good point. Do you mean vector legalization or integer legalization?"~~ "Interesting idea. I wasn't sure whether you meant integer or vector legalization. Vector legalization seemed the better fit, so I tried it. Is this what you had in mind?"

Social Currency

Shared trust makes the review process easier.

- Explain your choices.
- Follow through on feedback and regressions.
- Help review other changes.

"goodwill is our currency."

LLVM Code-Review Policy

Leave a Useful Review

"You do not need to be an expert in some area of the code base to review patches"

LLVM Code-Review Policy: Non-Experts Should Review Code

- Review NFC (no functional change) PRs.
- Review PRs in the same area as your own contributions.
- When leaving a review, you can:
  - Explain what you checked; defer approval when appropriate.
  - Suggest a concrete alternative and explain why.
  - Ask: "Why not take this approach?"
  - Give a qualified approval: "LGTM, but please wait for an expert reviewer's approval before landing."

Takeaways, Resources, and Thanks

- Prepare a change reviewers can evaluate.
- Bring concrete changes to each round of review.
- Review other people's code, too.

Contributing to LLVM

LLVM Developer Policy

LLVM Code-Review Policy and Practices

LLVM AI Tool Use Policy

LLVM Testing Infrastructure Guide: Precommit workflow for tests

InstCombine contributor guide

So you want to change the LLVM-IR?

Thank you to Jeremy Morse for inspiring this talk, and to Aidan Goldfarb, Drew Kersnar, Yonah Goldberg, Meredith Julian, and Robert Virany for helping prepare it.

- Slides: justinfargnoli.github.io/slides/contributing.pdf

Help Us Shape the Future

US LLVM Developers' Meeting 2026

Scan the QR code to connect with us.
NVIDIA is the engine of the world's AI infrastructure. We are the world leader in accelerated computing.

Check out our open career opportunities here:
https://www.nvidia.com/en-us/about-nvidia/careers/

